

MCS3301 Introduction to Matlab for Control

Dr Ado Haruna
Dept. of Mechatronics Engineering
Bayero University Kano
aharuna.mct@buk.edu.ng

Matlab for Control Introduction

- **Introduction**
- **MATLAB Basics**
- **Vectors and Matrices**
- **Scripts and Plots**
- **Calculus**

Introduction

- The Matlab name is derived from the term MATrix LABoratory.
- Matlab is a programming environment developed by Mathworks.
- Matlab is used in solving a wide range of mathematical and engineering problems.
- Matlab has a well written and organised help which is the primary reference source.
- The simplest way of accessing help is to use the pull down menu or press F1.
- Help for Matlab keywords can also be accessed by typing `help <keyword>` at the command prompt

Matlab Environment

- The primary way of interacting with Matlab is at the command window. The user can directly enter commands at the prompt >>
- The History window shows previously entered Matlab commands.
- The Current Folder window shows the current directory Matlab is working in.
- The Workspace window shows and allows variables to be managed.

Matlab Variables

- The A variable is essentially a tag that is assigned to a value while that value remains in memory.
- Matlab variables must begin with a letter which may be followed by a combination of numbers and letters.
- Variables in Matlab are case sensitive so 'A' and 'a' are not the same variable.
- Variables are basically either numeric, string or symbolic.
- Numeric variables have a numeric value which may be complex
e.g

>> a = 5 assigns a value of 5 to the variable 'a'

>> h = 4 + 3i assigns the complex quantity to h

Matlab Variables

- The A variable is essentially a tag that is assigned to a value while that value remains in memory.

String variables hold alphanumeric characters which have no numeric value
e.g.

```
>> Name = 'MECHATRONICS'
```

- Symbolic variables are a third type of variable can be manipulated according to the usual rules of mathematics.
- The `syms` keyword is used in Matlab to declare a symbolic variable
e.g.

```
>> syms x y
```

```
>> x + x + y
```

```
ans =
```

```
2*x + y
```

- A semi-colon is used to suppress the output after assigning a value to a variable
- The `clear` command is used to clear a variable from memory.
- Use the `whos` command to check variables in the workspace.

Arithmetic Functions & Expressions

Arithmetic Expressions

<code>a = 5</code>	<code>% Assign 5 to a</code>
<code>b = 7</code>	<code>% Assign 7 to b</code>
<code>c = a+b</code>	<code>% Add two numbers</code>
<code>d = a-b</code>	<code>% Subtract two numbers</code>
<code>e = a*b</code>	<code>% Multiply two numbers</code>
<code>f = a/b</code>	<code>% Divide two numbers</code>
<code>g = a^2</code>	<code>% Square a number</code>

Arithmetic Functions

<code>a = sin(pi/2)</code>	<code>% Sin using radians</code>
<code>b = abs(-2.5)</code>	<code>% Absolute value</code>
<code>c = sqrt(5)</code>	<code>% Square root</code>
<code>d = exp(3.1)</code>	<code>% Exponential</code>
<code>e = floor(3.2)</code>	<code>% Largest integer less than 3.2</code>
<code>f = rem(7,5)</code>	<code>% Remainder after division</code>

Vectors and Matrices

- One of the strengths of Matlab is the ease of creating and manipulating vectors and matrices (**MAT**rix **LAB**oratory).

```
A = ones(5,5)           % 5*5 matrix of ones
B = rand(5,5)           % 5*5 matrix of random values [0,1]
C = [1 2; 3 4]          % 2*2 matrix
I = eye(5)              % 5*5 identity matrix
a = ones(5,1)           % Column vector of 5 ones
b = zeros(1,5)          % Row vector of 5 zeros
c = 3:7                 % Row vector [3 4 5 6 7]
d = [-1 1 4]            % Row vector with 3 values
```

- You can access the individual elements in an array or vector:

```
e = B(1,2)              % Get value in 1st row, 2nd column
f = c(3)                 % Get value in 3rd element
```

- The `find()` function can be used to find indices of elements in a vector. Eg `>>ind = find(C>3)` creates a vector with the indices of the elements in `C > 3`.
- Operations that act on vectors and matrices can often be written in a single line of Matlab (Vectorization) instead of loops thus making Matlab code more efficient.

Vectors and Matrices

- Given a square matrix B

```
D = B'           % Matrix transpose
inv(B)           % Inverse of a matrix
det(B)           % determinant of a matrix
eig(B)           % eigenvalue of a matrix
```

- The dot operator is used to perform element-wise calculation of a vector or Matrix. Eg

```
e = c.^2         % Square each element in vector c
```

- Other vector and matrix operators include:

```
e = length(c)    % Length of a vector
[r,c] = size(B)  % Matrix size (rows & columns)
e = min(c)        % Minimum of a vector
e = max(c)        % Maximum of a vector
e = sum(c)        % Sum of vector elements
e = prod(c)       % Product of vector elements
e = mean(c)       % Mean of a vector
e = diag(B)       % vector of diagonal elements of B
e = norm(c)       % norm of a vector
```

Solving Algebraic Equations

- The roots command may be used to solve (find the roots) of an equation having a single unknown e.g. to solve the expression

$$x^2 - 5x + 6 = 0$$

```
>> roots([1 -5 6])
```

```
ans =
```

```
3.0000
```

```
2.0000
```

- Alternatively, the solve command may also be used i.e.

```
>> solve('x^2 -5*x +6 = 0')
```

- The expression for the solve command may also be symbolic i.e.

```
>> syms a b c x;
```

```
>> solve('a*x^2 + b*x + c')
```

```
ans =
```

```
-(b + (b^2 - 4*a*c)^(1/2))/(2*a)
```

```
-(b - (b^2 - 4*a*c)^(1/2))/(2*a)
```

Exercise: Simultaneous Linear Equations

- Use Matlab to solve the simultaneous equations
$$4x - 2y + 3z = 11$$
$$x + 4y + 2z = -12$$
$$3x + y - 5z = 8$$
- For a linear set of equations $Ax = B$, if A is square and invertible (non singular) then the solution can in Matlab be obtained as

```
>> x = inv(A)*B
```
- If the solution is correct then we should find that

```
>> norm(B-A*x) ≈ 0
```
- It is better and easier in Matlab to use the command

```
>> X = A\B
```
- In the case of a rectangular matrix (more rows than columns) the above gives the least squares solution.

```
>> A = [A;10 11 12]           % add one row to A
>> B = [B;10]                 % add one row to B
>> X = A\B                    % Least squares solution
```
- Because the linear equations are inconsistent you should find that $\text{norm}(B-A*x)$ is a non-zero value, which is a measure of the degree of inconsistency.

Matlab Scripts and Plots

- A Matlab script is a text file with a **.m** extension which contains Matlab commands.
- Rather than entering a sequence of Matlab commands into the command window, the most common way to use Matlab is to create short programs.
- Matlab scripts allow for programs to be saved and also to be used as functions
- To create a script use the `edit` command or click the **New** button and select **script** from the drop down menu or simply press **CTRL+N**.
- Press **F5** or the **Play** button in the editor to run the program.
- Comments (not executed) may be added using the `%` sign to the code to make it easier to follow.

Matlab Plots

- Matlab makes plotting data relatively easy.
- The `plot` function is used in Matlab to produce 2D plots. The following script plots $y = \sin(x)$ from 0 to 5π in steps of 0.1;

```
x = 0:0.1:5*pi;  
y = sin(x);  
plot(x,y)
```

- Several modifications like axis text, min/max values, display markers, font size colour etc can be easily made to Matlab plots. Eg Labels:

```
xlabel('x')  
ylabel('sin(x)')  
title('Plot of Sin(x) and Cos(x)')
```

- Use `help` to find commands to modify plots
- Plots can also be customised using the **Tools – Edit Plot** pull down menu.

Matlab Plots

- Use the `hold on` command to superimpose one plot on another. The following script plots $\sin(x)$ and $\cos(x)$ on the same figure.

```
x = 0:0.1:5*pi;  
y1 = sin(x);  
y2 = cos(x);  
figure(2)  
plot(x,y1,'-.r', 'linewidth',2) % red dashed-dotted  
hold on  
plot(x,y2,'--b','linewidth',2) % blue dashed
```

- The `subplot` command can be used to divide the figure window into an m by n grid.

```
clear all  
x = 0:0.1:5*pi;  
y1 = sin(x);  
y2 = cos(x);  
figure(2)  
subplot(2,1,1)  
plot(x,y1,'-.r', 'linewidth',2)  
subplot (2,1,2)  
plot(x,y2,'--b','linewidth',2)
```

Calculus

- Matlab can compute differentials and integrals of symbolic variables
- The `diff` statement is used to obtain differentials of symbolic variables in Matlab. For example

```
>> syms x;  
>> y = sin(x);  
>> diff(y)  
ans =  
cos(x)
```

- Differentiation can also be done with respect to a particular variable in a multivariate function.

```
>> syms x t  
>> y = sin(x*t^2)  
y =  
sin(t^2*x)
```

```
>> diff(y,x)  
ans =
```

```
t^2*cos(t^2*x)
```

```
>> diff(y,t)
```

```
ans =
```

```
2*t*x*cos(t^2*x)
```

Calculus

- The `int` statement is used to obtain integrals of symbolic variables in Matlab.

```
>> syms x
```

```
>> y = sin(x)
```

```
y =
```

```
sin(x)
```

```
>> int(y)
```

```
ans =
```

```
-cos(x)
```